

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5

8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I i.jaswanth (192525345) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: i.jaswanth

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, wellstructured documentation with diagrams, code,	Organized documentation with necessary	Basic documentation with missing	Poor or incomplete documentation

		and results	details	details	
--	--	-------------	---------	---------	--

1A:

For the relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

Assume:

- Each **SID** identifies one student.
- Each **CID** identifies one course.
- A student can enroll in many courses.
- A course can have many students.
- A student's **Grade** is specific to a particular student-course combination.

Functional Dependencies

The main functional dependencies are:

1. **SID → SName**
A student ID determines the student's name.
2. **CID → CName**
A course ID determines the course name.
3. **CID → Instructor**
A course ID determines its instructor.
4. **(SID, CID) → Grade**
The combination of student ID and course ID determines the grade obtained.

Therefore, the complete set can be written as:

F = { SID → SName, CID → CName, CID → Instructor, (SID, CID) → Grade }

Candidate Key

Consider **(SID, CID)**:

- SID determines SName.
- CID determines CName and Instructor.
- SID + CID determines Grade.

So:

(SID, CID)+ = {SID, SName, CID, CName, Instructor, Grade}

Hence, **(SID, CID)** determines all attributes.

Neither **SID** nor **CID** alone can determine all attributes, so the candidate key is:

Candidate Key: (SID, CID)

Functional Dependency	Meaning
SID → SName	Student ID determines Student Name
CID → CName	Course ID determines Course Name
CID → Instructor	Course ID determines Instructor
SID, CID → Grade	Student-Course combination determines Grade

Final Answer:

Functional Dependencies: SID → SName, CID → CName, CID → Instructor, (SID, CID) → Grade

Candidate Key: (SID, CID)

```
mysql> SELECT * FROM STUDENT_COURSE;
+----+-----+----+-----+-----+-----+
| SID | SName | CID | CName          | Instructor | Grade |
+----+-----+----+-----+-----+-----+
| 101 | Arun  | 201 | DBMS           | Kumar     | A     |
| 101 | Arun  | 202 | Operating Systems | Ravi      | B     |
| 102 | Priya | 201 | DBMS           | Kumar     | A     |
| 102 | Priya | 202 | Operating Systems | Ravi      | A     |
| 103 | Rahul | 203 | Computer Networks | Suresh    | B     |
+----+-----+----+-----+-----+-----+
5 rows in set (0.01 sec)
```

2A:

Demonstrate the Process of Converting an Unnormalized Relation into 1NF

Consider the following **unnormalized relation**:

STUDENT(SID, SName, Courses)

Step 1: Unnormalized Form (UNF)

In the unnormalized relation, a student can have **multiple courses in a single field**.

SID	SName	Courses
101	Arun	DBMS, OS
102	Priya	DBMS, CN
103	Rahul	OS

Here, the **Courses** attribute contains multiple values. Therefore, it is **not in 1NF**.

Step 2: Identify the Problem

The relation violates **First Normal Form (1NF)** because:

- Courses contains multiple values.
- Each field should contain only **one atomic value**.
- Repeating groups are present.

Step 3: Convert into 1NF

Separate each course into an individual row.

SID	SName	Course
101	Arun	DBMS
101	Arun	OS
102	Priya	DBMS
102	Priya	CN
103	Rahul	OS

Now, every attribute contains a **single atomic value**.

Step 4: Determine the Key

A student can take multiple courses, so SID alone cannot uniquely identify a row.

The combination: **(SID, Course)**

uniquely identifies each record.

Therefore:

Candidate Key = (SID, Course)

Step 5: Justification for 1NF

The resulting relation is in **1NF** because:

1. All attributes contain atomic values.
2. There are no repeating groups.
3. Each row represents a single student-course combination.
4. The combination (SID, Course) uniquely identifies each row.

Final 1NF Relation

STUDENT_CO

URSE(SID, SName, Course)

Candidate Key: (SID, Course)

Thus, the unnormalized relation is converted into **1NF by eliminating multi-valued attributes and ensuring that every field contains a single atomic value**.

```
mysql> SELECT * FROM STUDENT_1NF;
+-----+-----+-----+
| SID | SName | Course |
+-----+-----+-----+
| 101 | Arun  | DBMS   |
| 101 | Arun  | OS     |
| 102 | Priya | CN     |
| 102 | Priya | DBMS   |
| 103 | Rahul | OS     |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

3A:

Given Relation

R(A, B, C, D, E)

Functional Dependencies:

- $A \rightarrow BC$
- $BC \rightarrow D$
- $D \rightarrow E$

Step 1: Find the Candidate Key

Calculate the closure of A:

$A^+ = \{A\}$

Using $A \rightarrow BC$:

$A^+ = \{A, B, C\}$

Using $BC \rightarrow D$: A^+

$= \{A, B, C, D\}$

Using $D \rightarrow E$: $A^+ =$

$\{A, B, C, D, E\}$

Therefore:

Candidate Key = A

Step 2: Check for Partial Dependencies

A relation is in **2NF** when it is already in 1NF and there are **no partial dependencies of nonkey attributes on a part of a composite candidate key**.

Here, the candidate key is A, which consists of only **one attribute**.

Therefore, **partial dependency is not possible**.

So, R is already in 2NF. Step

3: Dependency Analysis

The dependencies are:

FD Type

$A \rightarrow B, C$ Full dependency on key A

$BC \rightarrow D$ Dependency between non-key attributes

$D \rightarrow E$ Dependency between non-key attributes

There is **no partial dependency**, because the key contains only A.

Final Answer

Candidate Key: A 2NF

Relation:

R(A, B, C, D, E)

No decomposition is required to achieve **2NF**.

Note: The relation has a **transitive dependency** ($A \rightarrow D \rightarrow E$), so it is not in 3NF. A further decomposition would be needed for 3NF.

```
mysql> SELECT A, B, C, D, E
-> FROM R;
+----+-----+-----+-----+-----+
| A  | B    | C    | D    | E    |
+----+-----+-----+-----+-----+
| 1  | B1   | C1   | D1   | E1   |
| 2  | B2   | C2   | D2   | E2   |
| 3  | B3   | C3   | D3   | E3   |
+----+-----+-----+-----+
3 rows in set (0.00 sec)
```

4A:

-- 1. Create Original EMPLOYEE Table

```
CREATE TABLE EMPLOYEE (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT,  
    DeptName VARCHAR(50),  
    ManagerID INT  
);
```

-- 2. Insert Data

```
INSERT INTO EMPLOYEE VALUES  
(101, 'Arun', 10, 'Computer Science', 201),  
(102, 'Priya', 10, 'Computer Science', 201),  
(103, 'Rahul', 20, 'Mechanical', 202),  
(104, 'Sahil', 20, 'Mechanical', 202);
```

-- 3. Display Original Table

```
SELECT * FROM EMPLOYEE;
```

-- 4. Create DEPARTMENT Table for 3NF

```
CREATE TABLE DEPARTMENT (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50),  
    ManagerID INT  
);
```


-- 5. Insert Department Data

INSERT INTO DEPARTMENT VALUES

(10, 'Computer Science', 201),

(20, 'Mechanical', 202);

-- 6. Create EMPLOYEE_3NF Table

CREATE TABLE EMPLOYEE_3NF (

EmpID INT PRIMARY KEY,

EmpName VARCHAR(50),

DeptID INT,

FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)

);

-- 7. Insert Employee Data

INSERT INTO EMPLOYEE_3NF VALUES

(101, 'Arun', 10),

(102, 'Priya', 10),

(103, 'Rahul', 20),

(104, 'Sahil', 20);

-- 8. Display EMPLOYEE_3NF

SELECT * FROM EMPLOYEE_3NF;

-- 9. Display Department Details

```
SELECT * FROM DEPARTMENT;
```

-- 10. Join Both Tables

```
SELECT e.EmpID, e.EmpName, d.DeptID, d.DeptName, d.ManagerID
FROM EMPLOYEE_3NF e
JOIN DEPARTMENT d
ON e.DeptID = d.DeptID;
```

```
mysql> SELECT e.EmpID, e.EmpName, d.DeptID, d.DeptName, d.ManagerID
-> FROM EMPLOYEE_3NF e
-> JOIN DEPARTMENT d
-> ON e.DeptID = d.DeptID;
```

EmpID	EmpName	DeptID	DeptName	ManagerID
101	Arun	10	Computer Science	201
102	Priya	10	Computer Science	201
103	Rahul	20	Mechanical	202
104	Sahil	20	Mechanical	202

4 rows in set (0.00 sec)

5A:

BCNF Verification and Decomposition

Given relation:

R(A, B, C, D)

Functional Dependencies:

- **AB → C**
- **C → D**
- **D → A**

Step 1: Find Candidate Keys

AB⁺:

AB → C → D → A

Therefore:

AB⁺ = {A, B, C, D}

So **AB** is a candidate key.

BC⁺: C →

D → A

Therefore:

BC⁺ = {A, B, C, D} So

BC is a candidate key.

BD⁺:

D → A, and AB → C Therefore:

BD⁺ = {A, B, C, D}

So **BD** is also a candidate key.

Candidate keys are:

AB, BC, BD

Step 2: Check BCNF

For BCNF, for every non-trivial FD **X → Y**, **X must be a superkey**.

- **AB → C → AB** is a candidate key 
- **C → D → C** is **not** a superkey 
- **D → A → D** is **not** a superkey 

Therefore:

R is NOT in BCNF.

Step 3: BCNF Decomposition

Using the violating dependency **C → D**, decompose:

R₁(C, D) R₂(A, B, C)

For **R₁(C,D)**:

C → D, and C is a key of R₁. Therefore,
R₁ is in BCNF.

For **R₂(A,B,C)**:

AB → C, and AB is a key of R₂.
Therefore, **R₂ is in BCNF.**

Final Answer

BCNF decomposition:

R₁(C, D)

$R_2(A, B, C)$

Both relations satisfy **BCNF**.

```
mysql> SELECT R2.A, R2.B, R2.C, R1.D
-> FROM R2
-> JOIN R1
-> ON R2.C = R1.C;
+---+---+---+---+
| A | B | C | D |
+---+---+---+---+
| 1 | 10 | 100 | 1000 |
| 1 | 20 | 200 | 2000 |
| 2 | 10 | 100 | 1000 |
| 2 | 30 | 300 | 3000 |
+---+---+---+---+
4 rows in set (0.00 sec)
```

6A:

Lossless-Join Decomposition Using Chase Algorithm

Consider the relation:

$R(A, B, C)$

Functional dependency:

$A \rightarrow B$

Decompose R into:

- $R_1(A, B)$
- $R_2(A, C)$

Step 1: Initial Chase Table

Row	A	B	C
R ₁	a	b	c ₁
R ₂	a	b ₂	c

Here, common attribute **A** has the same value a.

Step 2: Apply FD $A \rightarrow B$

Since both rows have the same value for **A**, the FD $A \rightarrow B$ forces their B values to become equal.

Row	A	B	C
R ₁	a	b	c ₁

R ₂	a	b	c
----------------	---	----------	---

Now the first row contains:

a, b, c and the second row

contains:

a, b, c

Step 3: Check the Chase Result

A row contains the distinguished symbols for **all attributes**: (**a,**

b, c)

Therefore, the decomposition is **lossless-join**.

Conclusion

$R(A,B,C) \rightarrow R_1(A,B), R_2(A,C)$ is a **lossless-join decomposition** because the Chase algorithm produces a row containing all distinguished values.

```
mysql> SELECT R1.A, R1.B, R2.C
-> FROM R1
-> INNER JOIN R2
-> ON R1.A = R2.A;
```

A	B	C
1	B1	C1
1	B1	C2
2	B2	C1
2	B2	C3

4 rows in set (0.00 sec)

7A:

4NF Decomposition of TEACHER

Given relation:

TEACHER(TID, Subject, Hobby)

Assume a teacher can independently have **multiple subjects** and **multiple hobbies**.

Step 1: Identify Multi-Valued Dependencies

The multi-valued dependencies are:

- **TID $\twoheadrightarrow$ Subject**
- **TID $\twoheadrightarrow$ Hobby**

This means the subjects and hobbies of a teacher are independent of each other.

Step 2: Problem in the Original Relation For example:

TID	Subject	Hobby
1	DBMS	Cricket
1	DBMS	Music
1	OS	Cricket
1	OS	Music

The combinations create unnecessary redundancy.

Step 3: Decompose into 4NF

Using the MVD **TID $\twoheadrightarrow$ Subject**, decompose the relation into:

TEACHER_SUBJECT(TID, Subject)

and

TEACHER_HOBBY(TID, Hobby)

Step 4: Final 4NF Relations

TEACHER_SUBJECT

TID	Subject
1	DBMS
1	OS

TEACHER_HOBBY

TID	Hobby
1	Cricket

1	Music
---	-------

Conclusion

The original relation violates **4NF** because of the non-trivial multivalued dependencies:

TID $\twoheadrightarrow$ Subject

TID $\twoheadrightarrow$ Hobby

The 4NF decomposition is:

TEACHER_SUBJECT(TID, Subject)

TEACHER_HOBBY(TID, Hobby)

```
mysql>
mysql> SELECT * FROM TEACHER_HOBBY;
+-----+-----+
| TID | Hobby |
+-----+-----+
| 1   | Cricket |
| 1   | Music  |
| 2   | Reading |
+-----+-----+
3 rows in set (0.00 sec)
```

8A:

5NF Decomposition Using Join Dependencies

Consider the relation:

SUPPLIER_PART_PROJECT(Supplier, Part, Project)

Suppose a supplier supplies a particular **part** for a particular **project**.

Step 1: Original Relation

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S2	P1	J1

If the relationships between **Supplier, Part, and Project** are independent, storing all three together can create redundancy.

Step 2: Join Dependency

The relation can be represented using three smaller relations:

- **SP(Supplier, Part)**
- **SJ(Supplier, Project)**
- **PJ(Part, Project)**

The join dependency is: **R**

= **SP ⋈ SJ ⋈ PJ**

Step 3: Decomposition

SP(Supplier, Part)

Supplier	Part
S1	P1
S1	P2
S2	P1

SJ(Supplier, Project)

Supplier	Project
S1	J1
S1	J2
S2	J1

PJ(Part, Project)

Part	Project
P1	J1
P1	J2
P2	J1

Step 4: Why Redundancy Occurs

In the original three-attribute relation, information about the same supplier-part or supplierproject relationship may be repeated across multiple rows.

For example, if **S1 supplies P1** for several projects, S1-P1 is repeated for each project.

The decomposition stores each independent relationship only once.

Step 5: 5NF Relations

The final decomposition is:

SP(Supplier, Part)

SJ(Supplier, Project)

PJ(Part, Project)

These relations are in **5NF (Project-Join Normal Form)** when every non-trivial join dependency is implied by the candidate keys.

Conclusion

Join dependencies can cause redundancy when a relation contains several independent many-to-many relationships. Decomposing the relation into smaller relations removes this redundancy while allowing the original relation to be reconstructed through joins.

```

-- AND S3.Project = P3.Project;
+-----+-----+-----+
| Supplier | Part | Project |
+-----+-----+-----+
| S1       | P1   | J1      |
| S1       | P1   | J2      |
| S1       | P2   | J1      |
| S2       | P1   | J1      |
+-----+-----+-----+
4 rows in set (0.00 sec)

```

9A:

College Database: Anomalies and 3NF Normalization

Consider the poorly designed relation:

COLLEGE(StudentID, StudentName, DeptID, DeptName, CourseID, CourseName, Instructor)

1. Functional Dependencies

Assume:

- $\text{StudentID} \rightarrow \text{StudentName}, \text{DeptID}$
- $\text{DeptID} \rightarrow \text{DeptName}$
- $\text{CourseID} \rightarrow \text{CourseName}, \text{Instructor}$

There are transitive dependencies:

StudentID $\rightarrow$ DeptID $\rightarrow$ DeptName

2. Anomalies

Insertion Anomaly

- A new department cannot be stored unless at least one student is enrolled.
- A new course cannot be added unless a student is registered for it.

Deletion Anomaly

- If the last student in a department is deleted, the department information may also be lost.
- If the last student taking a course is deleted, course information may be lost. **Update**

Anomaly

- If a department name changes, it must be updated in multiple student records.
- Missing one update can create inconsistent data.

3. Normalize to 3NF

Decompose the original relation into:

STUDENT(StudentID, StudentName, DeptID)

DEPARTMENT(DeptID, DeptName) COURSE(CourseID, CourseName, Instructor)

If students can enroll in multiple courses, create:

ENROLLMENT(StudentID, CourseID)

4. Final 3NF Schema

STUDENT(StudentID, StudentName, DeptID)

DEPARTMENT(DeptID, DeptName)

COURSE(CourseID, CourseName, Instructor)

ENROLLMENT(StudentID, CourseID)

Conclusion

The original college schema contains **insertion, deletion, and update anomalies** due to redundancy and transitive dependencies. Decomposing it into **STUDENT, DEPARTMENT, COURSE, and ENROLLMENT** removes these anomalies and produces a **3NF database design**.

StudentID	StudentName	DeptName	CourseName	Instructor
102	Priya	Computer Science	Operating Systems	Kumar
101	Arun	Computer Science	DBMS	Ramesh
103	Rahul	Mechanical	Thermodynamics	Suresh
104	Sahil	Mechanical	DBMS	Ramesh

10A:

Compute Attribute Closures and Candidate Keys

Relation: R(A, B, C, D, E)

Functional Dependencies (FDs):

- $A \rightarrow BC$
- $B \rightarrow D$
- $CD \rightarrow E$

1. Compute A^+

Start with:

$$A^+ = \{A\}$$

Using $A \rightarrow BC$:

$$A^+ = \{A, B, C\}$$

Using $B \rightarrow D$: $A^+ =$

$$\{A, B, C, D\}$$

Using $CD \rightarrow E$: $A^+ = \{A,$

$$B, C, D, E\}$$

Therefore:

$$A^+ = \{A, B, C, D, E\}$$

Since A^+ contains all attributes, **A is a candidate key**.

2. Compute B^+

Start:

$$B^+ = \{B\}$$

Using $B \rightarrow D$:

$$B^+ = \{B, D\}$$

No other FD can be applied.

Therefore: B^+

$$= \{B, D\}$$

Not a key.

3. Compute C^+

$$C^+ = \{C\}$$

No FD can be applied.

Therefore:

$$C^+ = \{C\}$$

Not a key.

4. Compute D^+

$$D^+ = \{D\}$$

No FD can be applied.

Therefore:

$$D^+ = \{D\}$$

Not a key.

5. Compute E^+

$$E^+ = \{E\}$$

No FD can be applied.

Therefore:

$$E^+ = \{E\}$$

Not a key.

6. Check possible composite keys

Since **A alone is already a key**, any set containing A is not minimal.

Sets without A cannot determine A because **A never appears on the right-hand side of any FD**. Therefore, no set without A can be a candidate key.

Final Result

Attribute Set	Closure	Candidate Key?
A	{A, B, C, D, E}	Yes
B	{B, D}	No
C	{C}	No
D	{D}	No
E	{E}	No

Answer

The only candidate key is {A}.

Prime attribute: A

Non-prime attributes: B, C, D, E

```
+-----+-----+
| Attribute_Closure | Closure |
+-----+-----+
| B+                | {B, D} |
| C+                | {C}    |
| D+                | {D}    |
| E+                | {E}    |
+-----+-----+
4 rows in set (0.00 sec)
```